7. Qwen3-TTS 启动与运行调用链

适用当前 Qwen/Qwen3-TTS-12Hz-1.7B-Base：张量并行(Tensor Parallelism, TP)=1，三个阶段共用 pipeline 子进程，声码器配置 async_decode=false。保留核心调用，省略中间适配层。

第一页：主进程启动服务

```
run_server()
    │
    ├─ ① MultiProcessPipelineRunner(pipeline_config)
    │       → 保存流水线配置，创建运行器(Runner)
    │
    ├─ ② await mp_runner.start(timeout=startup_timeout)
    │       → prepare_pipeline_runtime(): 规划拓扑、GPU、端点
    │       → build_stage_groups(): 构建进程与阶段启动配置
    │       → Coordinator.start(): 启动主进程中的通信端点
    │       → create_task(Coordinator.run_completion_loop())
    │            → 后台接收阶段完成、流式输出和管理结果
    │       → StageGroup.spawn(): 启动 pipeline 子进程
    │            → run_process(): 创建并运行三个 Stage ← 第二页
    │       → StageGroup.wait_ready(): 等待子进程就绪
    │       → Coordinator.register_stage(): 注册三个阶段
    │       → create_task(monitor_children()): 监控子进程
    │
    ├─ ③ Client(coordinator) → create_app(client)
    │       → 创建 /v1/audio/speech 等 HTTP 接口
    │       → PipelineUvicornServer(uvicorn.Config(...))
    │       → create_task(warm_up_server()): 后台等待 HTTP 启动
    │       → serve_with_failure_watch(...)
    │            ├─ server.serve(): 监听 8000 端口，接收请求
    │            └─ mp_runner.wait_failed(): 等待流水线异常
    │
    └─ ④ finally → await mp_runner.stop()
            → 停止协调器与子进程，清理通信及运行资源
```

Coordinator 和 HTTP 服务位于主进程；三个 Stage 位于子进程。mp_runner.start() 返回时，阶段已就绪；HTTP 端口由后面的 server.serve() 开始监听。

当前 Qwen3-TTS 没有配置 HTTP 预热请求，warm_up_server() 只等待服务启动并标记就绪。模型初始化期间的 CUDA 图预热属于阶段构建流程。

第二页：子进程如何创建三个阶段

```
run_process(spec, ready_event, log)          ← pipeline 子进程
    │
    ├─ ① 遍历 spec.stage_specs，依次 construct_stage(spec)
    │       │
    │       ├─ construct_scheduler(spec)：按工厂路径创建调度器
    │       │       │
    │       │       ├─ preprocessing：第一个构建
    │       │       │       → create_preprocessing_executor()
    │       │       │       → ThreadedSimpleScheduler
    │       │       │             → 绑定 preprocess_qwen3_tts_payload()
    │       │       │
    │       │       ├─ vocoder：第二个构建
    │       │       │       → create_vocoder_executor()
    │       │       │       → 加载 Qwen3TTSTokenizer 音频编解码器
    │       │       │       → Qwen3TTSStreamingVocoderScheduler
    │       │       │             → warmup_now()：当前异步解码关闭，跳过
    │       │       │
    │       │       └─ tts_engine：第三个构建
    │       │               → create_sglang_tts_engine_executor()
    │       │               → Qwen3TtsEngineBuilder.build()
    │       │                     → 加载 Qwen3TTSTalker 及共享预处理上下文
    │       │                     → 初始化 KV 缓存、CUDA 图等推理资源
    │       │                     → Qwen3TTSModelRunner + OmniScheduler
    │       │
    │       └─ Stage(..., scheduler=scheduler)：包装通信与路由
    │
    ├─ ② local_dispatcher.register_many(stages)：注册进程内路由
    └─ ③ asyncio.run(_start_and_run())：启动并运行阶段 ← 第三页
```

构造顺序为 **preprocessing → vocoder → tts_engine**。先让声码器权重等常驻资源占用 GPU 显存(GPU Memory)，生成引擎再据此确定 KV 缓存(KV Cache)预算。

工厂返回调度器(Scheduler)，`construct_stage()` 再创建 `Stage`。同进程配置下，预处理可使用引擎构建时建立的共享上下文，不会在第一个工厂里重复加载整套前端。

第三页：阶段怎样持续运行、请求怎样流转

```
_start_and_run()                              ← 三个 Stage 已构造完成
    │
    ├─ ① 依次 await stage.start()
    │       → control_plane.start() + comm.start(): 启动通信
    │       → Thread("scheduler-阶段名").start(): 独立调度器线程
    │            → scheduler.warm_up_serving_thread()
    │            → scheduler.start(): 进入常驻调度循环
    │
    ├─ ② ready_event.set(): 向主进程报告就绪
    │
    └─ ③ create_task(stage.run()) × 3 → asyncio.gather(...)
            │                              ← 共用事件循环(Event Loop)
            ├─ control_plane.recv() → handle_message(): 处理控制消息
            ├─ drain_outbox(): 持续取出调度器输出，发送结果
            └─ abort_listener(): 监听请求中止
```

`Stage.run()` 开头再次调用 `start()` 时会直接返回，不会重复创建线程。普通阶段间载荷可通过 `LocalStageDispatcher` 在进程内直达目标阶段；就绪后，各阶段并发等待和处理消息。

```
HTTP 请求 → Client → Coordinator → preprocessing
    │
    ├─ ① ThreadedSimpleScheduler: 输入队列(Inbox) → 线程池
    │       → preprocess_qwen3_tts_payload(): 准备文本与参考音频提示
    │       → 输出队列(Outbox) → Stage.drain_outbox() → tts_engine
    │
    ├─ ② OmniScheduler: 接收请求 → waiting_queue → 组批
    │       → Qwen3TTSModelRunner: 模型前向、采样，生成音频编码
    │       → Outbox → Stage.drain_outbox() → vocoder
    │
    └─ ③ Qwen3TTSStreamingVocoderScheduler: 累积编码数据块
            → 编解码器(Codec)解码成波形(Waveform)
            → Outbox → Stage.drain_outbox() → Coordinator
            → Client → HTTP 音频响应
```

请求顺序为 **preprocessing → tts_engine → vocoder**。生成与解码可按数据块衔接；流式能力不等于 `async_decode`，当前关闭的是声码器内部的异步解码工作线程。

源码：服务入口、进程与阶段创建、阶段运行、Qwen3-TTS 工厂。